

evolving-graphrag

Incremental maintenance of a knowledge-graph RAG index under document churn: provenance-first retraction, selective community re-summarisation, and a measured cost-versus-freshness trade-off against full-reindex and append-only baselines.

REPORT DATE	VERSION	CONFIG HASH	SEED
06 September 2026	v0.1.0	8b1da1efb99381b1	1337
REPOSITORY	TESTS	EVAL RUNTIME	STATUS
github.com/Hrithik373/ evolving-graphrag	133 passing	2.2 s, reproducible	P0–P5 complete

1 Summary

The system is functionally complete against the plan: all six build phases are implemented, the evaluation harness runs end to end, and the headline result holds. Under document deletion the index serves **zero stale answers at 8.5× lower cost than rebuilding**, while an append-only incremental index — the cheap alternative — serves stale answers **100%** of the time.

The contribution is the maintenance layer, not answer quality. When a document changes, the system touches the minimal affected subgraph and the minimal set of community summaries. Deletion is a provenance walk in $O(\text{affected subgraph})$, never a rebuild. Every entity, relation and summary is traceable to the chunks that produced it, and that single invariant is what makes retraction possible at all.

Table 1 — Build phase status against the acceptance criteria set out in the blueprint.

PHASE	SCOPE	ACCEPTANCE CRITERION	STATUS
P0	Schema, provenance, offline cycle	Ingest, query, correct provenance citations	complete
P1	Churn engine, GC, dirty tracking	Deletion removes <i>exactly</i> the uniquely-supported subgraph	complete
P2	Async workers, selective recompute, API	Writes return fast; only dirty communities recomputed	complete
P3	Compaction, resolution, retrieval	Compaction re-clusters without rebuilding summaries	complete
P4	Eval harness, Pareto, observability	One command reproduces every table and figure	complete
P5	Deployment hardening, demo	<code>make up</code> brings up the stack; live churn demo	complete
—	Beyond plan: Postgres backend, deployment	Second store backend; free public deployment	added

2 Problem and approach

A GraphRAG index is expensive to build: every chunk is passed through an LLM for entity and relation extraction, and every detected community is summarised by a second LLM pass. The literature and available

implementations largely treat the index as immutable — when the corpus changes, the index is rebuilt. That is tolerable once and untenable for a corpus that receives edits and deletions continuously.

The naive incremental alternative is worse in a way that is easy to miss. An append-only index absorbs new documents cheaply but has no retraction path, so it continues to answer with facts whose only source has been deleted. It looks correct on every benchmark that does not test deletion.

The invariant

Every relation stores the *set* of chunk identifiers that support it. Deletion becomes a graph walk:

- 1. Find the deleted document's chunks.
- 2. Strip those identifiers from every relation's provenance set. If the set empties, the relation has no source in the corpus and is deleted; if support remains, it survives, weakened.
- 3. Detach those chunks from every entity's mention set. Zero mentions means orphaned, and the entity is collected.
- 4. Mark the affected communities dirty. Nothing else is touched.

The same principle extends to derived state: an entity's description is prose written out of a particular chunk, so it is stored keyed by chunk and re-derived whenever the mention set changes. Without that, an entity that survives a deletion keeps quoting a document that was retracted — a leak found by the deletion scenario during development.

Systems framing

The index behaves like an **LSM-tree**. Incremental writes are cheap and local: mutations patch the graph and mark communities dirty. A periodic background **compaction** re-runs global community detection to amortise the drift that a long chain of local decisions accumulates. Compaction reconciles new clusters against existing communities by membership overlap, so a community whose membership did not change keeps its identifier, its summary and its clean flag — otherwise compaction would cost exactly as much as a full reindex.

3 Evaluation design

The benchmark is a **time-split churn protocol** rather than a static question set: a base corpus builds the index, then updates and new documents arrive, then documents are deleted. Questions are asked at the point where each matters.

Table 2 — Evaluation scenarios. The corpus holds 5 base documents, 2 updated, 2 new and 2 deleted.

SCENARIO	ASKED WHEN	WHAT IT DETECTS
base-on-base	after the initial build	baseline answer quality
base-on-updated	after churn	historical stability — did keeping up with change break what worked?
new-on-updated	after churn	new-information recall — did the change actually land?
deletion	after deletion	stale-answer rate — is retracted information still served?
deletion-survivors	after deletion	over-deletion control — did it delete too much?

Why the fifth scenario exists. The first four can all be passed by a system that simply deletes its entire index. The survivors set asks for facts that a *surviving* document still supports and requires them to remain answerable. Deleting too much is exactly as wrong as deleting too little, and only this pair of measurements distinguishes correct retraction from indiscriminate removal.

Two properties of the benchmark are asserted by tests rather than assumed. Every fact the deletion scenario requires to disappear is *uniquely* supported by a deleted document — a fact a surviving document also states would score a correct system as stale. And the full-reindex baseline is denied this project's content-hash extraction cache, since crediting it with the optimisation under evaluation would understate the true cost of rebuilding.

4 Results

4.1 Deletion — where incremental indexes fail

Table 3 — Behaviour after the deletion set is removed. Stale-answer rate is the fraction of questions still answered with a fact only a deleted document supported. Survivor recall is the over-deletion control. Lower cost is better in both token columns.

SYSTEM	STALE-ANSWER RATE ↓	SURVIVOR RECALL ↑	DELETION TOKENS	CHURN TOKENS	ENTITIES
evolving (<i>this work</i>)	0.00	1.00	2 705	12 281	18
full reindex	0.00	1.00	22 866	52 193	18
append-only	1.00	1.00	0	12 787	27
flat vector RAG	0.00	0.33	0	599	0

Three distinct failure modes are visible. **Append-only** spends nothing and is wrong every time — it has no retraction path at all. **Flat vector RAG** deletes easily, because dropping a document means dropping its chunks, but cannot answer across documents: its survivor recall of 0.33 means two thirds of the facts that *should* have survived became unanswerable. **This system matches full reindex on both correctness measures** while spending 8.5× fewer tokens on the deletion phase and 4.2× fewer across the churn phase.

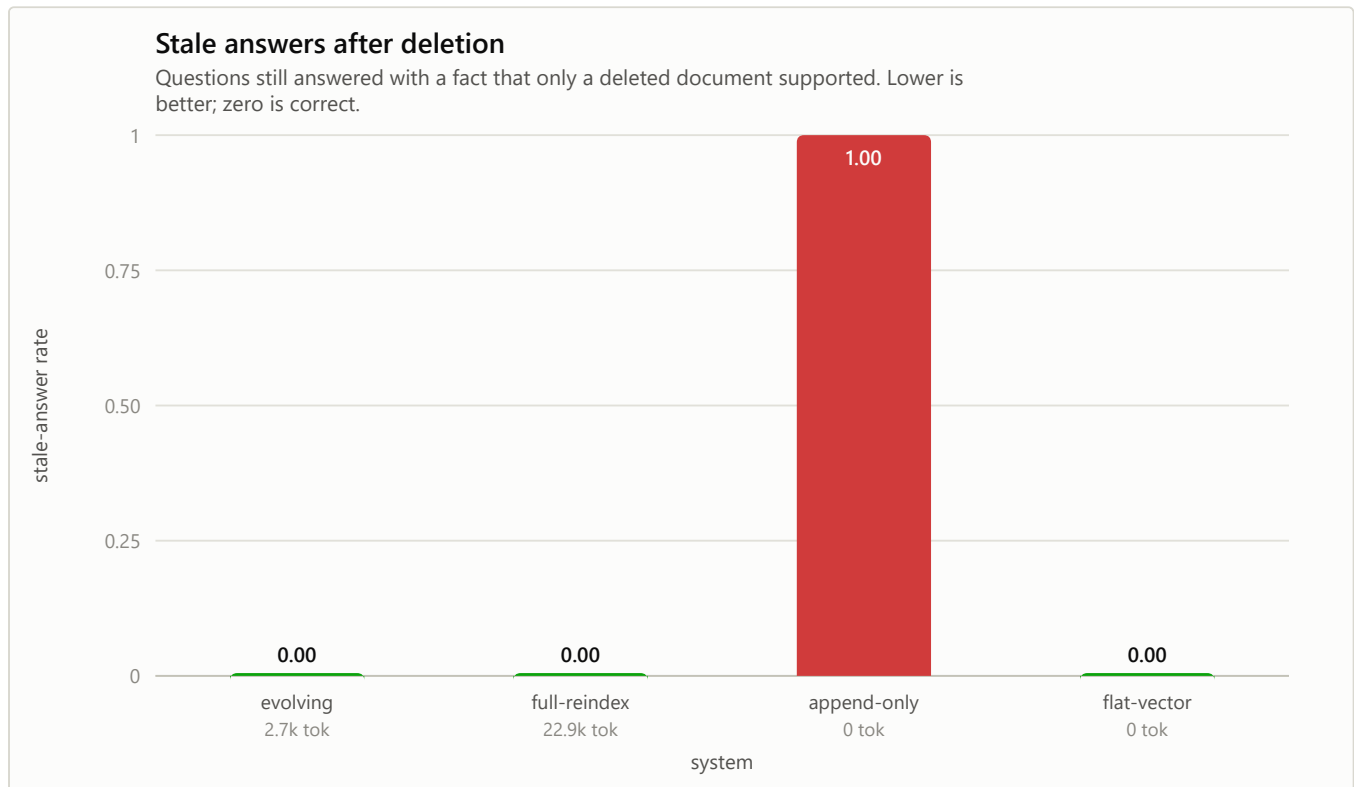


Figure 1 — Stale-answer rate after deletion, with the cost each system paid.

4.2 Answer quality is not traded away

Table 4 — Fact recall (and token F1, in grey) across the non-deletion scenarios. Cheap and fresh would be worthless if the answers had degraded.

SYSTEM	BASE-ON-BASE		BASE-ON-UPDATED		NEW-ON-UPDATED	
evolving (<i>this work</i>)	0.83	0.43	0.83	0.43	1.00	0.63
full reindex	0.83	0.43	0.83	0.43	1.00	0.63
append-only	0.83	0.43	0.83	0.43	1.00	0.63
flat vector RAG	0.67	0.24	0.67	0.24	1.00	0.27

The three graph systems are identical on every quality measure; they differ only in what maintenance costs. Flat vector RAG is materially worse on multi-hop questions, which is the expected cost of having no graph.

4.3 Cost versus freshness

The tunable is the **recompute budget**: how many dirty community summaries may be regenerated per document change.

Table 5 — The cost-versus-freshness curve. Full reindex is a reference point, not a sweep point.

CONFIGURATION	UPDATE TOKENS	SUMMARIES RECOMPUTED	STALE SUMMARIES / QUERY	DIRTY AFTER	SURVIVOR RECALL
budget 0	5 497	0	2.24	71%	1.00
budget 1	8 601	6	1.41	43%	1.00
budget 2	10 773	12	0.82	29%	1.00
budget 4	14 986	19	0.00	0%	1.00
drain	14 986	19	0.00	0%	1.00
<i>full reindex</i> (reference)	75 059	56	0.00	0%	1.00

Full freshness costs **5.0× fewer tokens than rebuilding** (14 986 against 75 059), and the budget can be traded down to **13.7× fewer** where some staleness is acceptable. Freshness is purchasable in small increments, and the price of the first increment is far below the price of a rebuild.

A finding worth stating precisely. Notice which freshness measure responds to the budget. The *stale-answer rate* is a property of the *graph*: once provenance GC is correct it is zero at every budget, because retraction never depended on summaries in the first place. What the budget actually buys is *reader-visible* staleness — how many out-of-date community summaries a query leans on. Conflating the two would have produced a flat and meaningless curve.

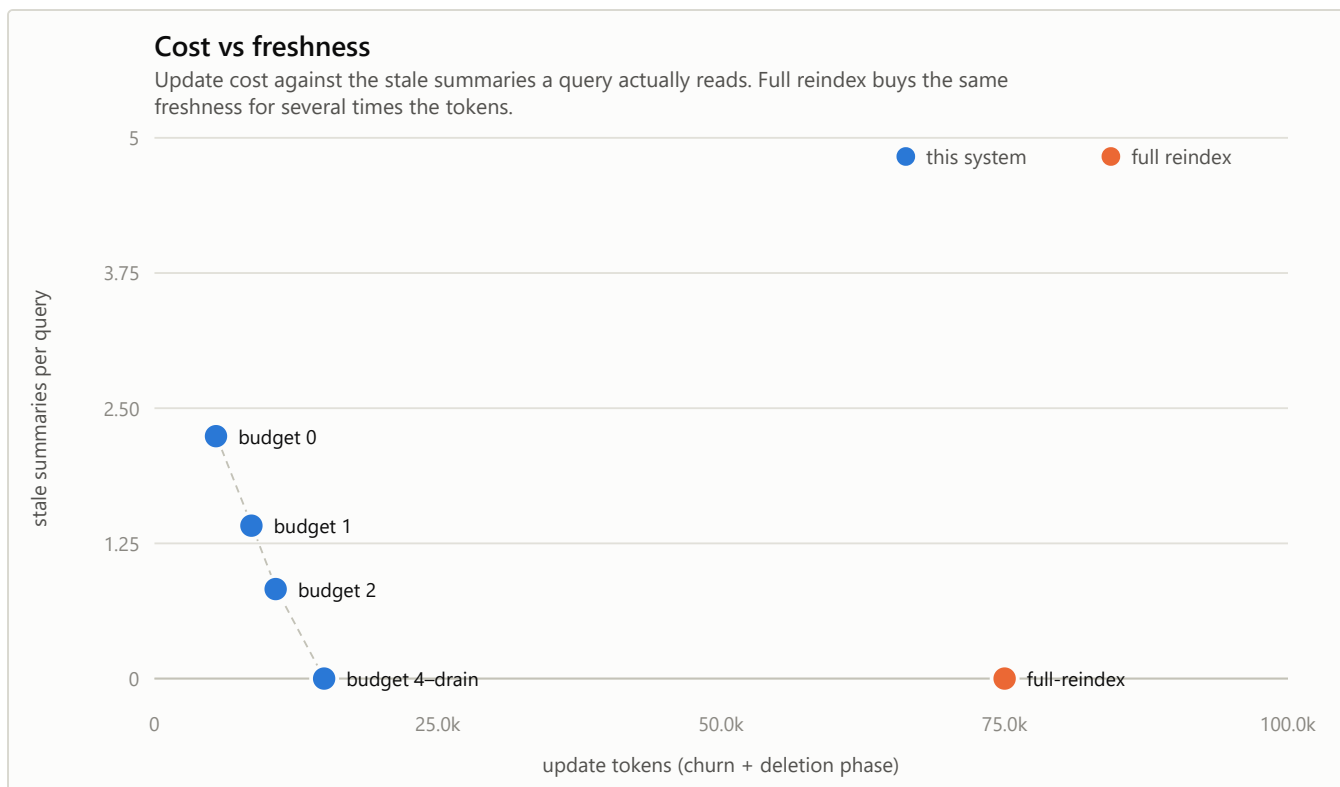


Figure 2 — Update cost against reader-visible staleness. The dashed line is the attainable frontier; full reindex sits far to the right at the same freshness.

4.4 Scaling — the asymptotic argument

Table 6 — Index size and cost as the corpus is replicated with disjoint entities.

DOCUMENTS	ENTITIES	RELATIONS	REBUILD TOKENS	ONE UPDATE	REBUILD ÷ UPDATE
5	20	38	14 995	2 491	6×
10	37	75	33 209	2 495	13×
20	71	149	70 005	2 499	28×
40	137	295	148 122	2 503	59×

This is the strongest result in the report. The cost of one document change is **flat** — 2 491 tokens at five documents and 2 503 at forty, a variation under one percent — while the cost of rebuilding grows linearly with the corpus. The advantage is therefore not a constant factor but one that **grows without bound**: from 6× to 59× across an eight-fold increase in corpus size. Extrapolating the same relationship, a corpus of a few thousand documents would make rebuilding three to four orders of magnitude more expensive per change.

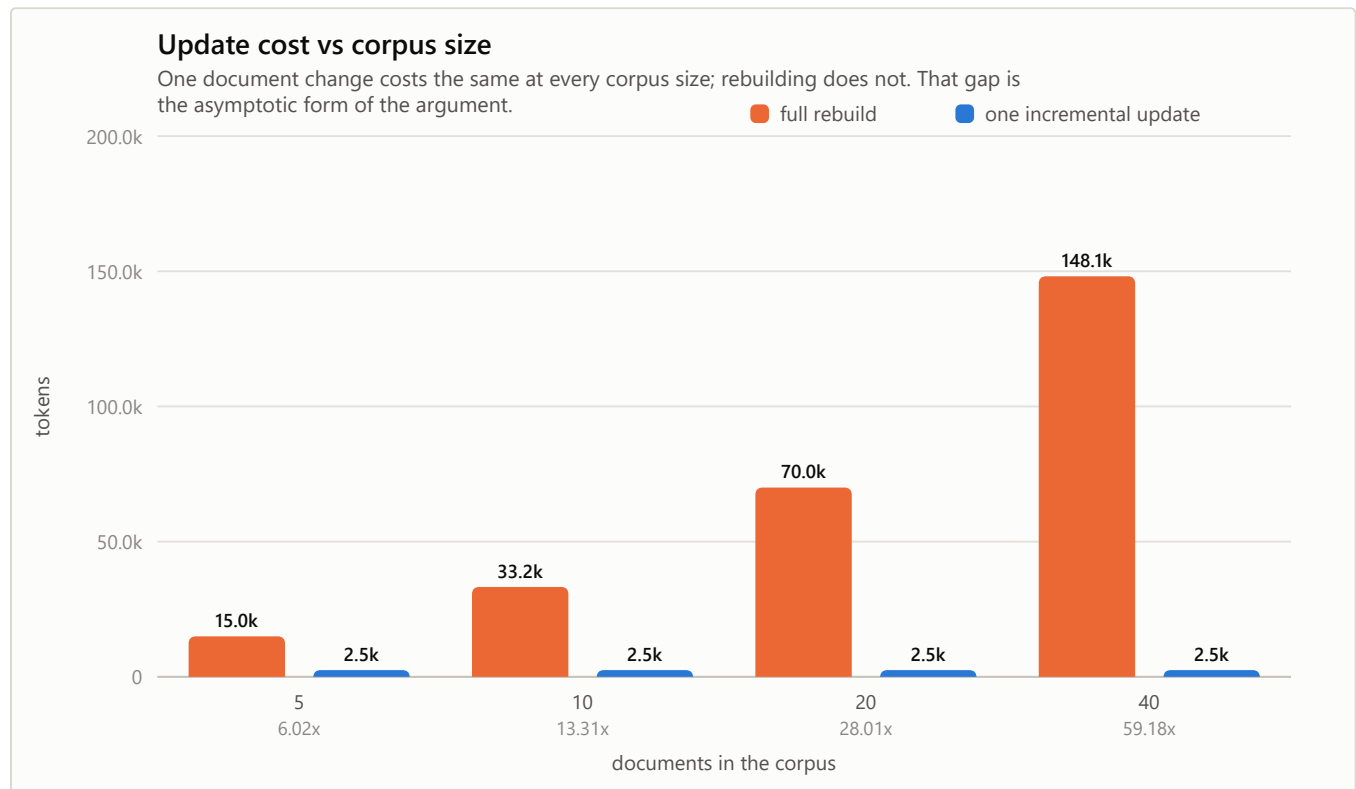


Figure 3 — Rebuild cost grows with the corpus; incremental update cost does not.

4.5 Compaction ablation — a negative result

Table 7 — Quality and cost with compaction at various intervals (in changes).

INTERVAL	COMPACTIONS	UPDATE TOKENS	COMMUNITIES	MEAN SIZE	BASE RECALL	STALE RATE
off	0	14 986	7	2.6	0.83	0.00
every 1	6	23 419	3	6.0	0.83	0.00
every 2	3	18 914	3	6.0	0.83	0.00
every 4	1	16 630	5	3.6	0.83	0.00
every 8	0	14 986	7	2.6	0.83	0.00

On this corpus, compaction costs tokens and buys nothing measurable. Running it after every change raises update cost from 14 986 to 23 419 — a 1.6× increase — while base recall and stale-answer rate are unchanged. It does consolidate the graph into fewer, larger communities (7 → 3), which is the structural effect it is designed to have.

The honest interpretation is that the corpus is too small and the change stream too short for incremental placement drift to accumulate to the point where it degrades retrieval. The mechanism is implemented and demonstrably works; the experiment that would justify it needs a larger corpus and a longer churn sequence. Reporting it as a positive result would be unsupported by this data.

4.6 Entity-resolution sweep — an inconclusive experiment

Table 8 — Sweeping the embedding-similarity merge threshold.

T _{SIM}	ENTITIES	RELATIONS	MERGE RATE	BASE RECALL
0.60	19	37	0.627	0.83
0.70	19	37	0.627	0.83
0.80	19	37	0.627	0.83
0.86	19	37	0.627	0.83
0.92	19	37	0.627	0.83
0.98	19	37	0.627	0.83

The sweep is **completely flat**: every threshold from 0.60 to 0.98 produces identical entity counts and merge rates. This is a defect in the experiment, not a finding about the system. Nearly every merge on this corpus is decided by the exact canonical-key path, which runs before the similarity threshold is ever consulted, so τ_{sim} never binds. Making this experiment meaningful requires a corpus containing genuine surface-form variation — abbreviations, aliases, inconsistent naming — which the current synthetic corpus deliberately lacks. This is listed under next steps.

5 Engineering quality

5.1 Testing

133 tests, all passing. The churn engine carries **property-based tests** rather than examples alone, because "correct under churn" is a claim about arbitrary interleavings of add, update and delete:

- **Provenance closure** — nothing exists in the graph without a live chunk supporting it.
- **Deletion minimality** — deleting a document removes exactly what it uniquely supported.
- **Idempotence** — replaying a change never alters the graph a second time.
- **Order independence** — the final graph depends on the surviving document set, not on the order the churn arrived in.

5.2 Reproducibility

Every evaluation run is seeded and stamped with a `config_hash` derived from the entire settings object, and a seeded run is **bit-reproducible across processes** for every metric except wall-clock. This is enforced by a regression test, and reaching it exposed a real defect: the recompute queue had been ordered by timestamp, and two communities marked dirty within the same clock tick tie — so oldest-first alternated between time order and insertion order between runs. Under a bounded recompute budget that changed *which* summaries were regenerated, moving the Pareto curve. The ordering key is now an explicit counter, a total order by construction.

5.3 Multiple store backends, one contract

Three implementations of the same `GraphStore` interface: an in-memory store (the semantics oracle, used by tests and the harness), PostgreSQL, and ArcadeDB. A parity suite runs every test once per backend and requires identical behaviour.

This suite earned its keep. Running it against a live ArcadeDB for the first time found four defects invisible to code review: a DDL comment containing a semicolon was split by the schema loader and its tail parsed as SQL; `DELETE VERTEX` requires a `FROM` clause; `LIST OF FLOAT` rejects Python floats, which are doubles; and `SELECT ... ORDER BY` with an implicit projection returns one phantom row per storage bucket — eight copies of a single record. The last would have silently corrupted the dirty-community recompute queue in production.

The PostgreSQL backend is notable beyond portability. The central operation of the system — *which relations does this chunk support?* — becomes an array-containment query against a **GIN index**, making retraction an index scan. The graph database answers the same question through a collection predicate whose support varies by version and whose fallback is a client-side scan of every relation. Provenance-first maintenance turns out to fit an inverted index better than a graph traversal.

5.4 Cost accounting and observability

No model call exists unless it produced a cost record; cache hits are recorded as zero-cost calls rather than absent ones, so the cost axis of every figure is complete by construction. Prometheus exports the dirty-community gauge, write-latency histogram, selective-recompute counters, token counters and index-size gauges, with a provisioned Grafana dashboard.

6 Deployment

Two shapes are supported, and the distinction is stated plainly in the documentation rather than glossed over.

- **Full stack** — six services under Docker Compose: API, arq worker, ArcadeDB, Redis, Prometheus, Grafana. This is the configuration that demonstrates the architecture, because it is the only one with a real queue and a separate worker. Scaling worker replicas is the throughput knob.
- **Single-container demo** — API and console in one image against managed PostgreSQL, deployable free. It has real persistence and the full provenance semantics, but with no worker there is no queue, so extraction runs inline and *the asynchronous write path the project is about is not exercised*. The deployment guide says so explicitly.

Also delivered: a React console (document churn with per-change receipts, upload, querying with provenance and freshness, a force-directed graph explorer, community and resolution audit views, maintenance controls) and a CI pipeline covering lint, the full suite on two Python versions, all three store backends against live servers, the dependency-free clustering fallback, image builds and the frontend.

7 Limitations

Stated plainly, because each one bounds a claim above.

- **The offline extraction backend caps answer quality.** The default backend is a real rule-based extractor and extractive reader, not a stub — it is what makes the maintenance semantics provable in CI with no API key. But token F1 against a reference sentence is depressed by it, and base recall sits at 0.83. The maintenance metrics — stale-answer rate, update cost, summaries recomputed — are backend-independent, and those are the project's claims.
- **The corpus is small and synthetic.** It is engineered so the invariants are *checkable* — some facts have exactly one supporting document, others have two — not to be representative. The scaling study replicates it eightfold, which is replication, not diversity.
- **Two experiments are currently uninformative** — the compaction ablation and the resolution threshold sweep, for the reasons given in §4.5 and §4.6.
- **Vector search is brute-force** over an in-process matrix in all three backends. Exact and reproducible at this scale; `pgvector` is the drop-in upgrade.
- **Single-node only.** Production would need workers under a horizontal autoscaler keyed on queue depth, store replication, and communities sharded across worker pools.
- **No authentication.** Appropriate for a controlled demo, and documented as a precondition before any public exposure.

8 Next steps

1. **A larger, non-synthetic corpus.** The single change that would most strengthen the results: it would make the compaction ablation informative, give the resolution sweep real surface-form variation to resolve, and extend the scaling curve past 40 documents.
2. **Run the evaluation against a real model backend** to report answer quality that is not capped by the offline extractor, alongside the existing maintenance metrics.
3. **Exercise the worker path end to end under load** — queue depth, backpressure, and worker-replica scaling — which the current single-node demo does not stress.
4. **Report update latency by phase** (synchronous write, extraction, summarisation) to substantiate the fast-path claim with a distribution rather than a single figure.

9 Reproducing these results

Every number and figure in this report comes from one command against the repository at config hash `8b1da1efb99381b1`, seed `1337`:

```
make install && make test && make eval
```

The evaluation completes in 2.2 seconds with no API key, no GPU and no running services — store backend `memory`, extraction backend `mock`, embeddings `hash`, clustering `leiden`, Python 3.12.10.

evolving-graphrag v0.1.0 · progress report generated 06 September 2026 · figures and tables derived from `results/summary.json` at config hash `8b1da1efb99381b1` · github.com/Hrithik373/evolving-graphrag